NeuroKinematics — Generic Robot Abstraction & Automated Robot Onboarding

Faz Konumu

Bu çalışma **Hybrid fazı tamamen tamamlanıp kabul edildikten sonra ve Studio fazına başlanmadan önce** gerçekleştirilmelidir.

Roadmap şu şekilde güncellenmelidir:

Foundations

↓

Core

↓

Hybrid

↓

Multi-Robot Generalization / Automated Robot Onboarding

↓

Studio

Bu fazın amacı yeni bir IK algoritması geliştirmek değildir.

Amaç, KUKA KR6 R900 sixx üzerinde doğrulanmış NeuroKinematics mimarisini robot-özel bağımlılıklardan ayırarak farklı seri manipülatörlerde yeniden yazılım geliştirmeden kullanılabilir hale getirmektir.

Studio fazı bu generic robot interface üzerine kurulmalıdır.

---

1. Ana Hedef

NeuroKinematics şu anda referans robot olarak:

KUKA KR6 R900 sixx

üzerinde geliştirilmiş ve doğrulanmıştır.

Bu faz sonunda kullanıcı desteklenen yeni bir

robotu sisteme eklediğinde NeuroKinematics kodunun yeniden yazılması gerekmemelidir.
Hedef kullanıcı akışı:

```text
Import Robot
      ↓
URDF / Robot Description
      ↓
Robot Parsing
      ↓
Structural Validation
      ↓
Kinematic Validation
      ↓
Numerical Solver Preparation
      ↓
NUMERICAL READY
      ↓
Dataset Generation
      ↓
Neural Training
      ↓
```

```
Hybrid Validation
      ↓
FULLY READY
```

Kullanıcının yeni robot için:

- FK kodu yazması,
- Jacobian kodu yazması,
- dataset generator yazması,
- yeni solver sınıfları oluşturması,
- training scriptlerini değiştirmesi

gerekmemelidir.
Yeni robot onboarding işlemi mümkün olduğunca konfigürasyon ve otomatik pipeline üzerinden yapılmalıdır.

---

2. Scope

İlk generic robot sürümü şu robot sınıfıyla sınırlandırılmalıdır:

Fixed-base serial manipulators

İlk hedef:

- 6 DOF
- revolute joint ağırlıklı
- açık kinematik zincir
- tek end-effector
- standart URDF tanımı

olmalıdır.

İlk sürümde aşağıdakiler zorunlu değildir:

- parallel robots
- delta robots
- SCARA
- mobile manipulators
- branched kinematic trees
- dual-arm systems
- closed-chain mechanisms

- continuum robots

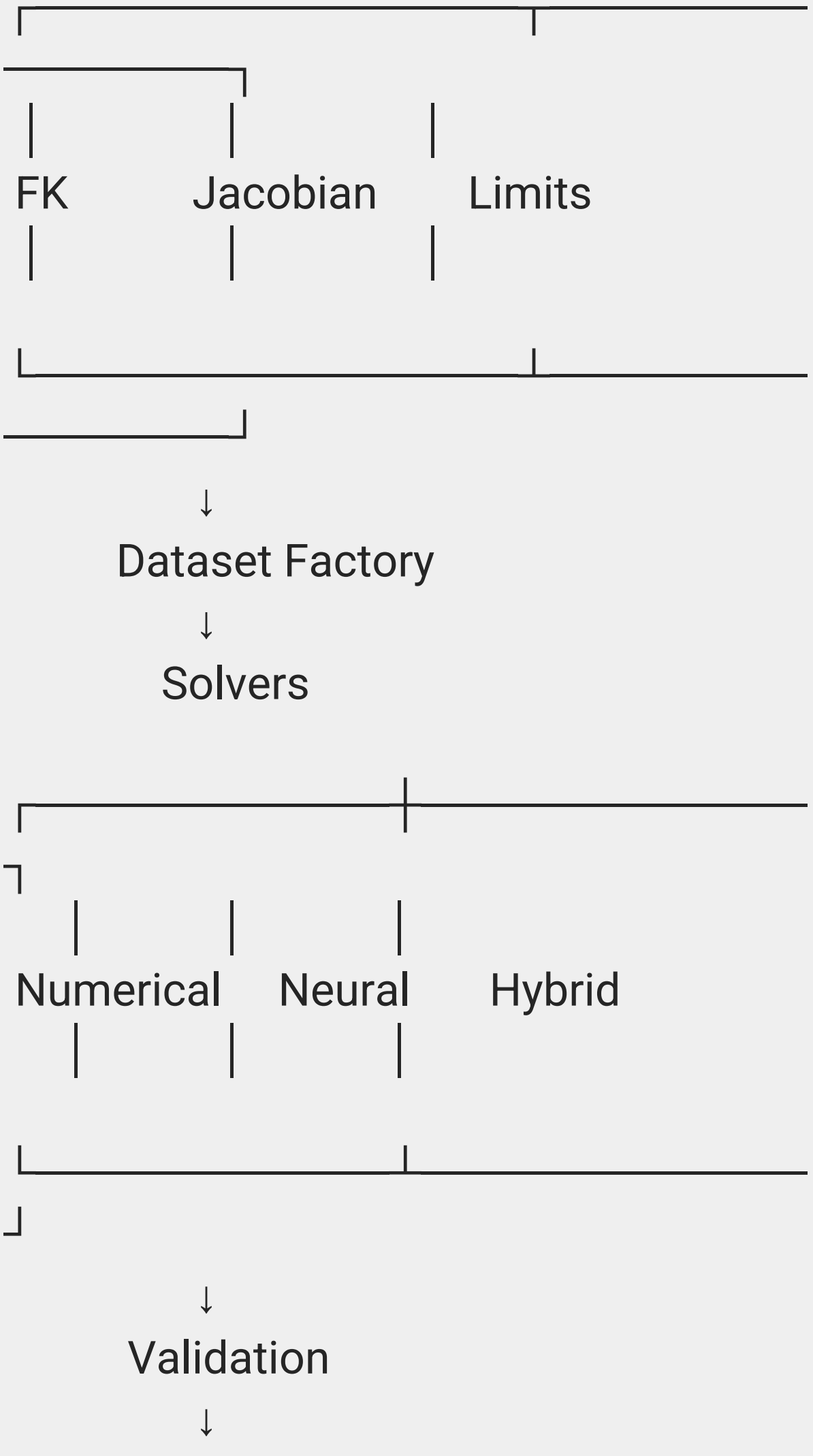
Mimari ileride genişletilebilir olmalıdır ancak ilk faz gereksiz şekilde genelleştirilmemelidir.

---

3. Temel Mimari Hedef

KUKA'ya özel kod ile NeuroKinematics algoritmaları birbirinden ayrılmalıdır.
Hedef mimari:

```text
Robot Description
        ↓
Robot Parser
        ↓
Generic Robot Model
        ↓
Robot Abstraction Interface
        ↓
```


FK
Jacobian
Limits
Dataset Factory
Solvers
Numerical
Neural
Hybrid
Validation

Studio

Hiçbir solver doğrudan:
KUKA link isimlerine,
KUKA joint isimlerine,
KUKA DOF sabitlerine,
KUKA-specific frame isimlerine
bağımlı olmamalıdır.

---

4. Generic Robot Model

Merkezi bir robot modeli oluştur veya mevcut robot representation varsa onu generic hale getir.
Örneğin:

```text
RobotModel

name
```

dof

joints
links

base_frame
end_effector_frame

joint_names
joint_types

joint_lower_limits
joint_upper_limits

velocity_limits
acceleration_limits

kinematic_chain

collision_geometry
visual_geometry

metadata

Bu model NeuroKinematics içerisinde robot hakkında tek gerçek kaynak olmalıdır.
Single Source of Truth prensibini koru.
Aynı joint limitlerini farklı config veya solver dosyalarında yeniden tanımlama.

---

5. Robot Import

İlk sürümde ana giriş formatı:
URDF
olmalıdır.
Varsa Xacro desteği opsiyonel olarak eklenebilir.
Parser aşağıdaki bilgileri çıkarmalıdır:

- robot name
- links
- joints
- joint types

- parent-child relations
- origins
- axes
- lower limits
- upper limits
- velocity limits
- visual meshes
- collision meshes
- base frame
- end-effector candidate
- DOF

Kullanıcının gerektiğinde:
base link
ve
end-effector link
seçebilmesine izin verilmelidir.

---

6. Structural Validation

Robot NeuroKinematics tarafından kabul edilmeden önce otomatik validation yapılmalıdır. En az şu kontroller uygulanmalıdır:

V-R01 — URDF validity

Dosya parse edilebiliyor mu?

V-R02 — Kinematic chain

Base ile end-effector arasında geçerli bir zincir var mı?

V-R03 — DOF

Desteklenen DOF aralığında mı?
İlk hedef:
6 DOF serial manipulator.

V-R04 — Joint types

Desteklenmeyen joint türü var mı?

V-R05 — Joint limits

Tüm controllable joint'ler için geçerli limitler mevcut mu?

V-R06 — Frame integrity

Transform zinciri tutarlı mı?

V-R07 — Duplicate / missing names

Joint ve link isimleri benzersiz mi?

V-R08 — End effector

Geçerli bir end-effector frame tanımlanabiliyor mu?
Her kontrol:

PASS
 WARNING
 FAIL
durumlarından birini vermelidir.
FAIL durumunda robot hazırlanma sürecine devam etmemelidir.

---

7. Kinematic Backend

Yeni robot için kullanıcıdan FK veya Jacobian kodu istenmemelidir.
Robot modelinden genel olarak:

```text
FK(q)
```

ve:

```text
J(q)
```

üretebilen bir backend kullanılmalıdır.
Bu backend mevcut NeuroKinematics FK/ Jacobian API sözleşmesiyle uyumlu olmalıdır.
KUKA için mevcut doğrulanmış implementasyon regression reference olarak korunmalıdır.
Generic backend sonucu KUKA referans implementasyonuyla karşılaştırılmalıdır.

---

8. Numerical Ready State

Yeni robot neural training tamamlanmadan önce mümkünse kullanılabilir hale gelmelidir.
Structural ve kinematic validation tamamlandığında:

```text
NUMERICAL READY
```

durumu oluştur.
Bu durumda şu özellikler çalışmalıdır:

- FK
- Jacobian
- DLS
- LM
- joint limit handling
- target pose solving
- basic validation

Bu önemli bir ürün gereksinimidir.
Kullanıcı neural model eğitiminin tamamlanmasını beklemeden robotu numerical solver ile kullanabilmelidir.

---

9. Automatic Dataset Generation

Mevcut Foundations dataset factory robot-independent hale getirilmelidir.
Yeni robot için otomatik:

```text
joint sampling
      ↓
FK
      ↓
pose generation
      ↓
workspace dataset
```

pipeline'ı çalışmalıdır.
Dataset üretimi robotun:
joint limits
ve
kinematic structure

bilgilerinden otomatik türetilmelidir.
Dataset generator içinde KUKA-specific değer kalmamalıdır.

---

10. Sampling Strategy

En az şu örnekleme grupları korunmalıdır:

Uniform samples

Joint space genel örnekleme.

Boundary samples

Joint limitlerine yakın örnekler.

Singularity-oriented samples

Mümkün olduğu ölçüde düşük manipulability

bölgeleri.

Validation samples

Training dışı bağımsız test seti.
Dataset boyutları config üzerinden değiştirilebilir olmalıdır.

---

11. Workspace Analysis

Yeni robot hazırlanırken otomatik workspace analizi yapılmalıdır.
Raporla:

- yaklaşık Cartesian workspace bounds
- reachable position distribution
- orientation distribution
- manipulability distribution

- joint usage statistics
- boundary density

Mümkünse artifact olarak:

```text
workspace_3d.png
workspace_xy.png
workspace_xz.png
manipulability_histogram.png
```

üret.

---

12. Neural Model Preparation

KUKA için eğitilmiş neural modelin doğrudan farklı robotlarda kullanılabileceği varsayılmamalıdır.

Yeni robot için otomatik:

```text
Robot
 ↓
Dataset
 ↓
Training Config
 ↓
Training
 ↓
Validation
 ↓
Neural Model
```

pipeline'ı çalıştır.
Neural architecture mümkün olduğunca ortak kalmalıdır.
Robot başına yeni Python model sınıfı oluşturma.
Değişmesi gerekenler config ve data üzerinden gelmelidir.

---

13. Hybrid Solver Preparation

Hybrid fazında geliştirilen:

```text
Neural prediction
        ↓
Numerical refinement
        ↓
Validated IK solution
```

pipeline'ı generic RobotModel üzerinden çalışmalıdır.
Yeni robot için ayrı hybrid solver yazılmamalıdır.
Aynı HybridSolver API kullanılmalıdır.

---

14. Robot Preparation Pipeline

Merkezi bir orchestration layer oluştur.

Örneğin:

```text
RobotOnboardingPipeline
```

veya mevcut mimariye uygun eşdeğer yapı.

Pipeline adımları:

```text
IMPORT
↓
PARSE
↓
STRUCTURAL VALIDATION
↓
KINEMATIC VALIDATION
↓
```

```
NUMERICAL BENCHMARK
↓
NUMERICAL READY
↓
DATASET GENERATION
↓
NEURAL TRAINING
↓
NEURAL VALIDATION
↓
HYBRID VALIDATION
↓
FULLY READY
```

Her aşama persistent status üretmelidir.

---

15. Readiness States

Robot için en az şu durumları tanımla:

```text
IMPORTED

VALIDATING

INVALID

NUMERICAL_READY

GENERATING_DATASET

TRAINING

VALIDATING_NEURAL

VALIDATING_HYBRID

FULLY_READY

FAILED
```

Status ile birlikte:

- current stage
- completed stages
- warnings
- failures
- metrics
- artifact locations

tutulmalıdır.

---

16. Kullanıcı Deneyimi Hedefi

Studio fazında kullanılmak üzere backend API oluştur.
Örneğin:

```text
add_robot(path)
```

```
prepare_robot(robot_id)

get_robot_status(robot_id)

get_robot_metrics(robot_id)

list_robots()

delete_robot(robot_id)
```

Studio daha sonra şöyle bir ekran gösterebilmelidir:

text

```
ROBOTS

KUKA KR6 R900 sixx
FULLY READY

CUSTOM-6
TRAINING
Dataset ✓
Numerical ✓
```

Neural 68%
Hybrid ○

ABB IRB120
NUMERICAL READY

Bu fazda tam GUI zorunlu değildir.
Ancak Studio'nun bunu doğrudan kullanabileceği temiz API oluşturulmalıdır.

---

17. Background / Long Running Jobs

Dataset generation ve neural training uzun işlemler olabilir.
Pipeline restart-resilient tasarlanmalıdır.
Bir işlem yarıda kaldığında mümkünse tamamlanan aşamalar tekrar çalıştırılmamalıdır.

Checkpoint mekanizması kullanılmalıdır.
Örneğin:

```text
Dataset generation ✓
Training checkpoint epoch 42
```

durumundan devam edilebilmelidir.

---

18. Time Expectations

Sistem kesin süre garantisi vermemelidir.
Örneğin:
“Robot 24 saat içinde hazır olur.”
gibi sabit bir ürün iddiası kullanılmamalıdır.
Hazırlık süresi:

- robot complexity

- dataset size
- CPU/GPU
- architecture
- training configuration

gibi faktörlere bağlıdır.
Ancak sistem:

```text
NUMERICAL READY
```

ve:

```text
FULLY READY
```

durumlarını ayırarak kullanıcının neural training tamamlanmadan da robota erişmesini sağlamalıdır.

---

19. Benchmark

Yeni robot tamamen READY olmadan önce standard benchmark çalıştırılmalıdır.
Benchmark en az:

- FK validation
- Jacobian validation
- numerical IK success rate
- position error
- orientation error
- convergence rate
- joint-limit violations
- neural IK metrics
- hybrid IK metrics
- runtime

içermelidir.

---

20. Ready Acceptance Criteria

Numerical Ready

Robot yalnızca aşağıdaki durumlarda NUMERICAL_READY olabilir:

- URDF PASS
- chain validation PASS
- FK validation PASS
- Jacobian validation PASS
- joint limits PASS
- numerical IK benchmark kabul sınırlarının üzerinde
- kritik regression yok

Fully Ready

FULLY_READY için:

- Numerical Ready PASS
- dataset integrity PASS
- neural model training complete
- neural validation complete
- hybrid solver validation PASS
- acceptance benchmark PASS
- artifact package generated

zorunlu olmalıdır.

---

21. Validation Robotları

Generic mimari yalnızca KUKA üzerinde test edilmemelidir.
En az üç robot kullan:

Robot A

KUKA KR6 R900 sixx
Regression reference.

Robot B

Farklı üreticiden yaygın bir 6DOF robot.
Örneğin:
UR5 / UR5e
veya
ABB IRB120

Robot C

Mümkünse basit custom 6DOF test robotu.
Amaç:
KUKA-specific varsayımların gerçekten ortadan

kaldırıldığını göstermek.

---

22. KUKA Regression

Bu fazın en kritik şartlarından biri:

KUKA performansını bozmamak.

Generic abstraction sonrası mevcut:

Foundations

Core

Hybrid

KUKA testlerinin tamamı tekrar çalıştırılmalıdır.

Önceki doğrulanmış sonuçlarla karşılaştır.

Kabul edilemez regression varsa faz PASS verme.

---

23. Robot Configuration Package

Her robot için standart paket oluştur.

Örneğin:

```text
robots/
    kuka_kr6/
        robot.urdf
        manifest.yaml
        config.yaml
        models/
        benchmarks/
        artifacts/

    custom_6dof/
        robot.urdf
        manifest.yaml
        config.yaml
        models/
        benchmarks/
        artifacts/
```

Ancak runtime mantığı robot klasörlerine özel kod içermemelidir.

---

24. Manifest

Her robot için machine-readable manifest üret. Örneğin:

```text
robot_name

robot_version

urdf_hash

dof

joint_names
```

base_frame

end_effector_frame

dataset_hash

model_hash

solver_versions

benchmark_results

readiness_status

created_at

Bu yapı reproducibility açısından zorunludur.

---

25. Model Registry

Robot ile neural modeli eşleştiren registry oluştur.
Örneğin:

```text
Robot:
CUSTOM-6

Neural Model:
nk_custom6_v1.onnx

Compatible Robot Hash:
abc123...

Dataset Hash:
def456...
```

Robot geometry değişirse eski model otomatik olarak güvenilir kabul edilmemelidir.
Hash mismatch varsa:

```text
MODEL INCOMPATIBLE
RETRAIN REQUIRED
```

durumu verilmelidir.

---

26. Robot Revision Handling

Fiziksel/custom robotlarda URDF zaman içinde değişebilir.
Örneğin link uzunluğu:
300 mm → 320 mm
olduğunda sistem bunu algılamalıdır.
URDF hash değişirse:

- previous validation invalidated
- dataset stale

- neural model stale
- hybrid validation stale

işaretlenmelidir.
Numerical solver yeni model üzerinden yeniden doğrulanmalıdır.

---

27. Error Handling

Kullanıcıya sadece:

```text
FAILED
```

verme.
Anlaşılır hata raporları üret.
Örneğin:

```text
Robot cannot be prepared.

Reason:
Joint 4 does not define an upper limit.

Required action:
Add a valid joint limit to the URDF.
```

veya:

```text
Kinematic validation failed.

Jacobian numerical agreement
exceeded tolerance.
```

---

28. Documentation

README ve roadmap güncellensin.
NeuroKinematics açıkça:
KUKA-specific IK project
olarak değil:
robot-independent inverse-kinematics research and engineering framework
olarak tanımlansın.
Ancak şu ayrım dürüst şekilde yazılmalıdır:

```text
Reference validated robot:
KUKA KR6 R900 sixx

Generic support:
Supported serial manipulators satisfying onboarding requirements.
```

---

29. Studio Dependency

Studio geliştirmesine bu faz tamamlanmadan başlanmamalıdır.
Studio hiçbir zaman doğrudan:

```text
KukaRobot
```

gibi bir implementation'a bağlanmamalıdır.
Studio sadece:

```text
RobotModel

RobotRegistry

SolverRegistry
```

RobotOnboardingPipeline

gibi generic interface'lerle konuşmalıdır.
Bu şart gelecekte çoklu robot kullanımının korunması için kritiktir.

---

30. Future Extensions

Bu faz sırasında aşağıdaki özellikleri implement etme ancak mimariyi engelleme:

- morphology-conditioned universal neural IK
- MAML / few-shot adaptation
- transfer learning
- arbitrary DOF networks
- motion planning
- collision-aware planning
- ROS2 hardware execution

- automatic physical calibration

Bunlar NeuroKinematics'in ileri sürümleri için tutulmalıdır.

---

31. Test Planı

En az aşağıdaki testleri oluştur:

T-MR01

KUKA URDF import.

T-MR02

İkinci üretici robot import.

T-MR03

Custom 6DOF robot import.

T-MR04

Invalid URDF rejection.

T-MR05

Missing joint limit rejection.

T-MR06

Generic FK correctness.

T-MR07

Generic Jacobian correctness.

T-MR08

Numerical IK success.

T-MR09

Automatic dataset generation.

T-MR10

Neural model training pipeline.

T-MR11

Hybrid solver integration.

T-MR12

Model/URDF hash mismatch detection.

T-MR13

Restart/checkpoint behaviour.

T-MR14

KUKA regression suite.

T-MR15

No KUKA-specific dependency in generic solvers.

---

32. Kod Denetimi

Repo üzerinde şu tür anti-patternleri ara:

```text
if robot == "kuka"

KUKA_JOINT_LIMITS

KUKA_DOF = 6

joint_1 ... joint_6 hardcode

kuka_fk()

kuka_dataset_generator()
```

Robot-specific değerlerin generic algorithm

katmanına sızmasını engelle.
Robot-specific metadata yalnızca robot description/config katmanında bulunmalıdır.

---

33. Git Süreci

Başlamadan önce:

```text
git status
git branch
git log -1
```

raporla.
Değişiklikleri mantıksal commitlere ayır.
Önerilen sıralama:

1. Generic RobotModel abstraction

2. URDF parser + structural validation
3. Generic FK/Jacobian backend
4. Numerical-ready pipeline
5. Generic dataset factory
6. Neural/hybrid onboarding
7. Registry + manifests + status system
8. Multi-robot tests
9. KUKA regression
10. Documentation

İlgisiz dosyalara dokunma.

---

34. Faz Kabul Kriterleri

Bu faz yalnızca aşağıdakilerin tamamı sağlandığında PASS olabilir:

- KUKA artık generic RobotModel üzerinden

çalışıyor.

- İkinci bir 6DOF robot aynı pipeline'da çalışıyor.
- Custom/test 6DOF robot aynı pipeline'da çalışıyor.
- Yeni robot için solver kodu yazılması gerekmiyor.
- URDF üzerinden numerical IK hazırlanabiliyor.
- Numerical Ready state çalışıyor.
- Dataset otomatik üretilebiliyor.
- Neural training robot-specific kod gerektirmiyor.
- Hybrid solver robot-independent çalışıyor.
- Robot/model compatibility hash ile doğrulanıyor.
- KUKA regression testleri PASS.
- Studio'nun kullanabileceği generic API mevcut.
- Yeni robot onboarding süreci dokümante edilmiş.

---

35. Fazın Nihai Başarı Tanımı

Bu fazın nihai başarısı şu senaryoyla ölçülmelidir:
Kullanıcı NeuroKinematics koduna dokunmadan:

```text
custom_robot.urdf
```

yükler.
Sistem robotu analiz eder.
Robot uygun ise önce:

```text
NUMERICAL READY
```

durumuna ulaşır.

Ardından otomatik dataset/training/validation süreci tamamlandıktan sonra:

```text
FULLY READY
```

durumuna ulaşır.
Kullanıcı bu noktadan sonra aynı NeuroKinematics API üzerinden:

```text
target pose
→ IK
→ validated joint configuration
```

işlemini yeni robot üzerinde gerçekleştirebilir.
Bu senaryonun çalışması bu fazın temel kabul testidir.

Nihai Rapor Formatı

Çalışma tamamlandığında:

Status

PASS / PARTIAL / FAIL

Architectural Changes

Generic Robot Model

Supported Robot Types

Robot Import Pipeline

Numerical Ready Results

Neural Ready Results

Hybrid Results

Multi-Robot Validation

Robot	Import	FK	Jacobian	Numerical	Neural	Hybrid	Status

KUKA Regression

Generated Artifacts

Known Limitations

Git

Readiness for Studio

Studio fazına geçiş yalnızca generic interface ve multi-robot acceptance criteria sağlandıysa önerilmelidir.